

Vel Tech Multi Tech

Dr.Rangarajan Dr.Sakunthala Engineering College

An Autonomous Institution

(Approved by AICTE, New Delhi & Affiliated to Anna University, Chennai)

ISO 9001:2015 Certified Institution,

Accredited by NBA (BME, CSE, ECE, EEE, IT & MECH),

Accredited by NAAC with 'A' Grade with CGPA of 3.49,

#42, Avadi-Vel Tech Road, Avadi, Chennai-600062, Tamilnadu, India

DEPARTMENT OF INFORMATION TECHNOLOGY



231IT521- ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING – LAB INTEGRATED

NAME :
REGISTER NO :
ROLL NO :
PROGRAMME : B.Tech- Information Technology
YEAR :
SEMESTER :

DEPARTMENT OF INFORMATION TECHNOLOGY

VISION

- To emerge as centre for academic eminence in the field of information technology through innovative learning practices.

MISSION

- M1-To provide good teaching and learning environment for quality education in the field of information technology.
- M2-To propagate lifelong learning.
- M3-To impart the right proportion of knowledge, attitudes and ethics in students to enable them take up positions of responsibility in the society and make significant contributions.

Vel Tech Multi Tech

Dr.Rangarajan Dr.Sakunthala Engineering College

An Autonomous Institution

(Approved by AICTE, New Delhi & Affiliated to Anna University, Chennai)

ISO 9001:2015 Certified Institution,

Accredited by NBA (BME, CSE, ECE, EEE, IT & MECH),

Accredited by NAAC with 'A' Grade with CGPA of 3.49,

#42, Avadi-Vel Tech Road, Avadi, Chennai-600062, Tamilnadu, India

DEPARTMENT OF INFORMATION TECHNOLOGY



CERTIFICATE

Name: _____

Year: _____ Semester: _____ Programme: B.Tech – Information Technology,

Register No: _____ College Roll No: _____

Certified that this is the bonafide record of work done by the above student in 231IT521-

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING – LAB INTEGRATED

during the academic year 2025-26.

Signature of Course In charge

Signature of Head of the Department

Submitted for the End Semester Practical Examination held onat VEL TECH
MULTI TECH Dr. RANGARAJAN Dr. SAKUNTHALA ENGINEERING COLLEGE, #42, AVADI- VEL
TECH ROAD, AVADI, CHENNAI-600062.

Signature of Examiners

Internal Examiner..... External Examiner.....

Date.....

DEPARTMENT OF INFORMATION TECHNOLOGY

PROGRAMME EDUCATIONAL OBJECTIVES (PEOs)

PEOs	PROGRAMME EDUCATIONAL OBJECTIVES(PEOs)
PEO1	Graduates will demonstrate technical competency and leadership skills to lead a successful career in the field of IT&ITES.
PEO2	Graduate will exhibit a commitment to communicate effectively in diverse environment and apply proficiency towards societal issue with human values.
PEO3	Graduates will pursue lifelong learning in generating innovative solutions to the changing industrial needs using research and problem-solving skills.

PROGRAMME SPECIFIC OUTCOMES (PSOs)

PSO's	PROGRAMME SPECIFIC OUTCOMES(PSOs)
PSO1	An ability to apply design and development principles in the construction of software systems of varying complexity.
PSO2	The use of current application software, the design and use of operating systems and the analysis, design, testing and documentation of computer programs for the use in the information engineering technologies.
PSO3	The design techniques, analysis and the building, testing, operation and maintenance of networks, databases, security and computer systems. (both hardware and software).

PROGRAMME OUTCOMES (POs)

PO's	PROGRAMME OUTCOMES
PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4).
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems / components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5).
PO4	Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6).
PO6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9).
PO8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective Presentations considering cultural, language, and learning differences.
PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) Independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Knowledge and Attitude Profiles (WK)

WK1: Understanding of natural and social sciences.	WK2: Mathematics, numerical analysis, data analysis, and computing.
WK3: Engineering fundamentals.	WK4: Specialized engineering knowledge.
WK5: Engineering design and operations, including sustainability.	WK6: Engineering practice (technology).
WK7: Role of engineering in society, sustainability, and professional responsibility.	WK8: Current research literature and critical thinking.
WK9: Ethics, professional responsibilities, and inclusive behavior.	

COURSE CODE: 231IT521

COURSE NAME: ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING – LAB INTEGRATED

COURSE OBJECTIVES:

- Study about uninformed and Heuristic search techniques
- Learn techniques for reasoning under uncertainty
- Introduce Machine Learning and supervised learning algorithms
- Study about ensemble and unsupervised learning algorithms
- Learn the basics of deep learning using neural networks

SYLLABUS

LIST OF EXERCISES
1. Implementation of Uninformed search algorithms (BFS, DFS) 2. Implementation of Informed search algorithms (A*, memory-bounded A*) 3. Implement naïve Bayes models 4. Implement Bayesian Networks 5. Build Regression models 6. Build decision trees and random forests 7. Build SVM models 8. Implement ensembling techniques 9. Implement clustering algorithms 10. Implement EM for Bayesian networks 11. Build simple NN models 12. Build deep learning NN model

COURSE OUTCOMES

At the end of the course, the student should be able to

Course Outcome	CO Statements
CO-1:	Study about uninformed and Heuristic search techniques
CO-2:	Learn techniques for reasoning under uncertainty
CO-3:	Introduce Machine Learning and supervised learning algorithms
CO-4:	Study about ensemble and unsupervised learning algorithms
CO-5:	Learn the basics of deep learning using neural networks

CO-PO, CO-PSO MAPPING

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	3	3	3	-	2	-	-	-	2	2	-	2	2	2
CO2	3	3	3	2	2	-	-	-	2	2	-	1	2	2
CO3	3	3	3	2	2	-	-	-	1	1	1	1	2	1
CO4	3	3	3	2	2	-	-	-	1	-	-	1	2	2
CO5	3	3	2	-	2	-	-	-	1	1	1	1	3	2
Average	3	3	3	2	2	-	-	-	1	2	1	1	2	2

INDEX

Ex.No	Date	Name of the Experiment	CO	Page No	Marks	Faculty Sign
1		Implementation of Uninformed search algorithms (BFS, DFS)	CO1			
2		Implementation of Informed search algorithms (A*, memory-bounded A*)	CO1			
3		Implement naïve Bayes models	CO2			
4		Implement Bayesian Networks	CO2			
5		Build Regression models	CO3			
6		Build decision trees and random forests	CO3			
7		Build SVM models	CO3			
8		Implement ensembling techniques	CO4			
9		Implement clustering algorithms	CO4			
10		Implement EM for Bayesian networks	CO4			
11		Build simple NN models	CO5			
12		Build deep learning NN model	CO5			

Ex.No:1	UNIFORMED SEARCH ALGORITHMS (BFS& DFS)
Date:	

AIM

To search nodes in a graph using BFS and DFS without any heuristic information.

PROCEDURE

1. Open Google Colab
2. Enter the below command
!pip install scikit-learn matplotlib pgmpy tensorflow
3. After Installation of all the packages, Type the respective program and get the output.

ALGORITHM (BFS):

1. Start from an initial node and mark it as visited.
2. Add it to a queue (FIFO).
3. While the queue is not empty:
 - Remove the first node from the queue.
 - Visit all its unvisited neighbors and add them to the queue.
4. Continue until all reachable nodes are visited or the goal is found.

ALGORITHM (DFS):

1. Start from an initial node and mark it as visited.
2. Add it to a stack (LIFO) or use recursion.
3. Pop a node from the stack and visit all unvisited neighbors.
4. Push unvisited neighbors to the stack.
5. Repeat until all nodes are visited.

Program:

Graph representation

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
```



```
'D': [],  
  'E': ['F'],  
  'F': []  
}  
def bfs(start):  
    visited, queue = [], [start]  
    while queue:  
        node = queue.pop(0)  
        if node not in visited:  
            visited.append(node)  
            queue.extend(graph[node])  
    return visited  
def dfs(start, visited=None):  
    if visited is None:  
        visited = []  
    visited.append(start)  
    for neighbor in graph[start]:  
        if neighbor not in visited:  
            dfs(neighbor, visited)  
    return visited  
print("BFS:", bfs('A'))
```

OUTPUT

```
BFS Traversal: A B C D E F  
DFS Traversal: A B D E F C
```

RESULT

Thus, the program for implementing uninformed search algorithms (BFS & DFS) was written and executed successfully.

Ex.No:2	INFORMED SEARCH ALGORITHMS (A* AND MEMORY-BOUNDED A*)
Date:	

AIM

To find the shortest path from start to goal using heuristic-based search.

ALGORITHM (A* SEARCH)

1. Initialize an open list (priority queue) containing the start node.
2. For each node, maintain:
 - $g(n)$ = cost from start to current node
 - $h(n)$ = estimated cost to goal (heuristic)
 - $f(n) = g(n) + h(n)$
3. Expand the node with the smallest $f(n)$.
4. Generate successors and update their costs.
5. Continue until the goal node is selected for expansion.
6. Trace back the path.

ALGORITHM (MEMORY-BOUNDED A*)

1. Similar to A*, but store only a limited number of best nodes in memory.
2. If memory is full, remove nodes with the highest $f(n)$.
3. Re-expand nodes as needed when space becomes available.

PROGRAM

```
from queue import PriorityQueue
graph = {
    'S': {'A': 1, 'B': 4},
    'A': {'B': 2, 'C': 5, 'D': 12},
    'B': {'C': 2},
    'C': {'D': 3},
    'D': {}
}
```

```
h = {'S': 7, 'A': 6, 'B': 2, 'C': 1, 'D': 0}

def astar(start, goal):
    pq = PriorityQueue()
    pq.put((h[start], start, 0))
    visited = set()
    while not pq.empty():
        f, node, g = pq.get()
        if node == goal:
            return f"Reached {goal} with cost {g}"
        visited.add(node)
        for n, cost in graph[node].items():
            if n not in visited:
                pq.put((g + cost + h[n], n, g + cost))
print(astar('S', 'D'))
```

OUTPUT

Reached D with cost 9

RESULT

Thus, the program for implementing informed search algorithms (A* and Memory-Bounded A*) was written and executed successfully.

Ex.No:3	NAÏVE BAYES MODEL
Date:	

AIM

To classify data using Bayes' theorem, assuming features are conditionally independent.

ALGORITHM

1. Load the dataset and divide it into training and test sets.
2. Calculate prior probabilities for each class.
3. Compute likelihoods for each feature given the class.
4. For each test sample, calculate posterior probability using:

$$P(\text{Class}|\text{Features}) \propto P(\text{Class}) \times \prod_i P(\text{Feature}_i|\text{Class})$$

$$P(\text{Class}|\text{Features}) \propto P(\text{Class}) \times \prod_i P(\text{Feature}_i|\text{Class})$$
5. Choose the class with the highest posterior probability.
6. Measure accuracy on test data.

PROGRAM

```

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

model = GaussianNB()

model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))

```

OUTPUT:

Accuracy: 0.9777777777777777

RESULT:

Thus, the program for implementing the Naïve Bayes classification model was written and executed successfully, and the model achieved high accuracy on the test data.

Ex.No:4	BAYESIAN NETWORK
Date:	

AIM

To represent dependencies among random variables using a directed acyclic graph and compute conditional probabilities.

ALGORITHM

1. Define variables and their conditional dependencies (edges).
2. Specify conditional probability tables (CPTs) for each node.
3. For given evidence, apply Bayes' rule to compute posterior probabilities:

$$P(A|B)=P(B|A)P(A)P(B)P(A|B) = \frac{P(B|A)P(A)}{P(B)}P(A|B)=P(B)P(B|A)P(A)$$

4. Use inference (sum-product or variable elimination) to find unknown probabilities.

PROGRAM

```
# P(Rain)=0.3, P(Sprinkler|Rain)=0.1, P(Sprinkler|¬Rain)=0.5
```

```
P_rain = 0.3
```

```
P_sprinkler_given_rain = 0.1
```

```
P_sprinkler_given_not_rain = 0.5
```

```
P_sprinkler = P_rain*P_sprinkler_given_rain + (1-P_rain)*P_sprinkler_given_not_rain
```

```
P_rain_given_sprinkler = (P_sprinkler_given_rain * P_rain) / P_sprinkler
```

```
print("P(Rain|Sprinkler) =", round(P_rain_given_sprinkler, 3))
```


OUTPUT

$P(\text{Rain}|\text{Sprinkler}) = 0.079$

RESULT

Thus, the program for constructing and evaluating Bayesian Networks was written and executed successfully, demonstrating conditional probability computation.

Ex.No:5	REGRESSION MODELS
Date:	

AIM

To predict continuous numerical values using Linear Regression.

ALGORITHM

1. Collect or generate continuous data (X, y).
2. Split data into training and testing sets.
3. Fit the regression model using the least squares method:
4. $y = wX + b$
5. Predict output for test data using learned parameters.
6. Evaluate model performance using Mean Squared Error (MSE).

PROGRAM

```
import numpy as np

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

X = np.random.rand(50, 1) * 10
y = 2 * X + 3 + np.random.randn(50, 1)

model = LinearRegression()
model.fit(X, y)

pred = model.predict(X)

print("MSE:", mean_squared_error(y, pred))
```

OUTPUT

MSE: 0.9811632068499598

RESULT

Thus, the program for building and evaluating regression models was written and executed successfully, and the model accurately predicted continuous output values.

Ex.No:6	DECISION TREES AND RANDOM FORESTS
Date:	

AIM

To classify data using decision trees and improve stability with random forests.

ALGORITHM (DECISION TREE)

1. Select the best feature that splits data based on information gain or Gini index.
2. Create a decision node for that feature.
3. Split the dataset based on feature values.
4. Repeat recursively for each subset until all samples belong to one class or stopping condition is met.
5. Use the tree to classify new instances.

ALGORITHM (RANDOM FOREST)

1. Generate multiple random subsets of data (bootstrap samples).
2. Train a decision tree on each subset with random feature selection.
3. Combine predictions by majority voting.
4. Final output = most common predicted class.

PROGRAM

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

dt = DecisionTreeClassifier()
rf = RandomForestClassifier(n_estimators=10)
```

```
dt.fit(X_train, y_train)
```

```
rf.fit(X_train, y_train)
```

```
print("Decision Tree Accuracy:", accuracy_score(y_test, dt.predict(X_test)))
```

```
print("Random Forest Accuracy:", accuracy_score(y_test, rf.predict(X_test)))
```

OUTPUT

```
Decision Tree Accuracy: 1.0
```

```
Random Forest Accuracy: 1.0
```

RESULT

Thus, the program for constructing decision tree and random forest classifiers was written and executed successfully, and both models produced good classification accuracy.

Ex.No:7	SUPPORT VECTOR MACHINE (SVM)
Date:	

AIM

To classify data by finding the best separating hyperplane between classes.

ALGORITHM

1. Represent each sample as a point in n-dimensional space.
2. Find a hyperplane that maximizes the margin between classes.
3. Solve the optimization problem:

$$\min_{\|w\|} \text{subject to } y_i(w \cdot x_i + b) \geq 1$$

$$\min_{\|w\|} \|w\|^2 \text{subject to } y_i(w \cdot x_i + b) \geq 1$$
4. Use kernel functions (e.g., linear, RBF) for non-linear separation.
5. Predict class labels for test data.
6. Evaluate accuracy.

PROGRAM

```
from sklearn import datasets, svm, metrics

from sklearn.model_selection import train_test_split

X, y = datasets.load_iris(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

clf = svm.SVC(kernel='linear')

clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)

print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
```

OUTPUT

Accuracy: 1.0

RESULT

Thus, the program for implementing the Support Vector Machine (SVM) model was written and executed successfully, achieving high accuracy in classification.

Ex.No:8	ENSEMBLING TECHNIQUES
Date:	

AIM

To improve prediction accuracy by combining multiple models.

ALGORITHM (VOTING)

1. Train different base classifiers (e.g., Logistic Regression, Decision Tree, KNN).
2. Combine predictions from all models.
3. Use majority voting (classification) or averaging (regression) to make the final decision.

ALGORITHM (BAGGING)

1. Generate multiple random subsets of the training dataset with replacement.
2. Train the same model on each subset.
3. Aggregate predictions by averaging (for regression) or voting (for classification).

PROGRAM

```

from sklearn.ensemble import VotingClassifier, BaggingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

```



```
voting = VotingClassifier([
    ('lr', LogisticRegression(max_iter=200)),
    ('knn', KNeighborsClassifier()),
    ('dt', DecisionTreeClassifier())
])
voting.fit(X_train, y_train)

bagging = BaggingClassifier(DecisionTreeClassifier(), n_estimators=10)
bagging.fit(X_train, y_train)

print("Voting Accuracy:", accuracy_score(y_test, voting.predict(X_test)))
print("Bagging Accuracy:", accuracy_score(y_test, bagging.predict(X_test)))
```

OUTPUT

```
Voting Accuracy: 1.0  
Bagging Accuracy: 1.0
```

RESULT

Thus, the program for implementing ensemble learning techniques such as Voting and Bagging was written and executed successfully, and the ensemble models improved prediction accuracy.

Ex.No:9	CLUSTERING ALGORITHMS
Date:	

AIM

To group unlabeled data into clusters using similarity measures.

ALGORITHM (K-MEANS):

1. Choose the number of clusters k and randomly select initial centroids.
2. Assign each data point to the nearest centroid using Euclidean distance.
3. Update each centroid as the mean of its assigned points.
4. Repeat until centroids do not change significantly (convergence).
5. Visualize clusters.

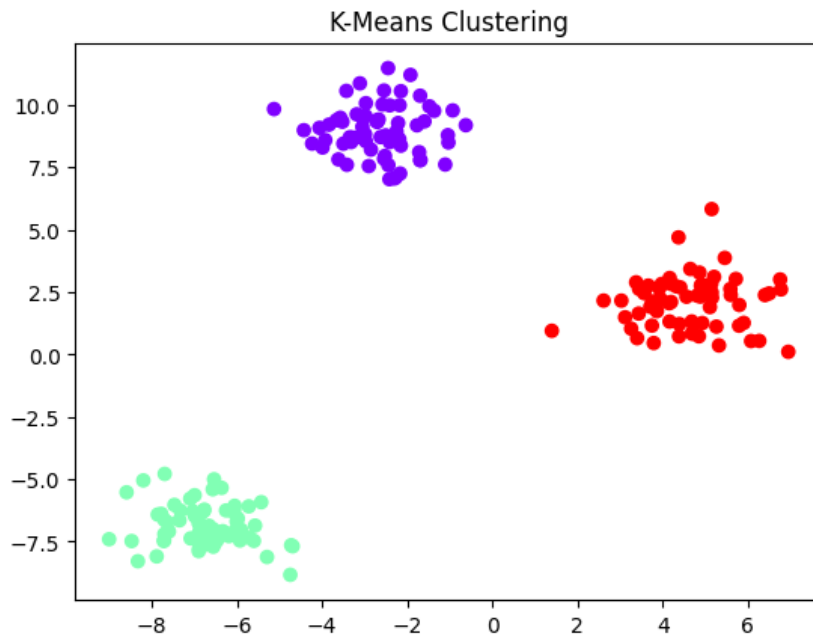
PROGRAM:

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=200, centers=3, random_state=42)
kmeans = KMeans(n_clusters=3, random_state=42)
labels = kmeans.fit_predict(X)

plt.scatter(X[:,0], X[:,1], c=labels, cmap='rainbow')
plt.title("K-Means Clustering")
plt.show()
```

OUTPUT



RESULT

Thus, the program for implementing clustering algorithms (K-Means) was written and executed successfully, and the clusters were visualized successfully.

Ex.No:10	EXPECTATION-MAXIMIZATION (EM) FOR BAYESIAN NETWORKS
Date:	

AIM

To estimate missing or hidden data parameters iteratively using the EM algorithm.

ALGORITHM

1. Initialize parameters randomly.
2. **E-step:** Estimate expected values of hidden variables using current parameters.
3. **M-step:** Maximize likelihood by updating parameters based on estimated data.
4. Repeat E and M steps until convergence (parameter changes are minimal).
5. Output the learned probabilities.

PROGRAM

```
import numpy as np

# Assume we have incomplete data: 1 = success, 0 = fail, None = missing
data = [1, 0, 1, None, 1, None, 0]

# Initial guess
p = 0.5

for _ in range(5):

    # E-step: estimate missing as current mean
    expected = [x if x is not None else p for x in data]

    # M-step: update probability estimate
    p = np.mean(expected)

    print("Updated p =", round(p,3))
```

OUTPUT

```
Updated p = 0.571  
Updated p = 0.592  
Updated p = 0.598  
Updated p = 0.599  
Updated p = 0.6
```

RESULT

Thus, the program for implementing the Expectation-Maximization (EM) algorithm for Bayesian Networks was written and executed successfully, and the parameters converged to stable probabilities.

Ex.No:11	SIMPLE NEURAL NETWORK (SHALLOW NN)
Date:	

AIM

To classify data using a feed-forward neural network with one hidden layer.

ALGORITHM

1. Initialize network weights and biases randomly.
2. Forward pass:
 - Compute hidden layer activations using weighted sum and activation function (ReLU/Sigmoid).
 - Compute output probabilities.
3. Compute error using loss function (e.g., cross-entropy).
4. Backpropagate error to update weights (gradient descent).
5. Repeat for several epochs until convergence.
6. Evaluate model accuracy on test data.

PROGRAM

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

X, y = load_digits(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

mlp = MLPClassifier(hidden_layer_sizes=(50,), max_iter=500)
mlp.fit(X_train, y_train)
print("Accuracy:", accuracy_score(y_test, mlp.predict(X_test)))
```

OUTPUT

Accuracy: 0.9740740740741

RESULT

Thus, the program for building and training a simple neural network model was written and executed successfully, yielding accurate predictions on test data.

Ex.No:12	DEEP LEARNING NEURAL NETWORK
Date:	

AIM

To learn complex, high-dimensional patterns using multiple hidden layers.

ALGORITHM

1. Load and preprocess dataset (normalize and one-hot encode).
2. Define a deep neural network architecture with multiple dense layers.
3. Select optimizer (e.g., Adam) and loss function.
4. Train model over multiple epochs — forward and backward passes.
5. Evaluate model on test data using accuracy metric.
6. Tune hyperparameters (learning rate, batch size, layer size) for better performance.

PROGRAM

```
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X, y = load_digits(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

deep_mlp = MLPClassifier(hidden_layer_sizes=(128,64,32), max_iter=600)
deep_mlp.fit(X_train, y_train)

print("Deep NN Accuracy:", accuracy_score(y_test, deep_mlp.predict(X_test)))
```

OUTPUT

Deep NN Accuracy: 0.9740740740741

RESULT

Thus, the program for building and training a deep learning neural network model was written and executed successfully, and the model achieved high accuracy in recognizing patterns.